

Hvordan bruke Argo CD ApplicationSet

Denne guiden vil hjelpe å ta i bruk ApplicationSet for deploy av Kubernetes-manifester uten å måtte konfigurere Application manuelt.

Introduksjon

Når man deployer applikasjoner med Argo CD [ApplicationSet](#) trenger man ikke konfigurere Argo CD Application -manifester manuelt. Disse genereres i stedet ut fra en mal med standard-konfigurasjon. Dette gjør at man unngår menneskelige feil og repetitiv boiler-plate.

Det lønner seg å ha en forståelse for hvordan Argo CD Application s fungerer når man skal ta i bruk ApplicationSet . Les mer i [Konfigurere Argo CD Application](#).

Denne guiden vil vise deg hvordan man setter opp applikasjoner ved hjelp av ArgoCD ApplicationSet .

Mer informasjon i Sopra Steria sin offisielle OpenShift-dokumentasjon: [Auto defined Argo CD Applications](#)

Forutsetninger

- Argo CD er satt opp som en del av tenant-bestillingen. Se eventuelt [Endre namespace](#).
- Argo CD har tilgang til git-repo med manifester. Se eventuelt [Tilgangstyring](#).
- Tenanten er konfigurert med enable_auto_defined_apps . Se eventuelt [Tenant-oppsettet](#).

```
argocd:
  enable_auto_defined_apps: true
```

- Du har grunnleggende forståelse for [Helm](#) og [Kustomize](#).
- Du har grunnleggende forståelse for [ArgoCD Applications](#).

Deploye applikasjoner med ApplicationSet

Når man bruker ApplicationSet vil Kubernetes-manifester som ligger i

`<basepath>/applicationsets/<miljø>/<applikasjon>/` automatisk bli deployet til OpenShift. At man har innhold i nevnt mappe i filstrukturen, fører til at ArgoCD Application -manifestet blir generert opp automatisk.

Deploy med Kustomize

Følg eksempler og beskrivelse i Sopra Steria sin OpenShift dokumentasjon: [Auto-defined ArgoCD applications - Kustomization file](#)

Deploy av Helm Charts

For å deploye Helm-charts ved hjelp av ApplicationSet anbefales det å bruke en kombinasjon av Helm og Kustomize.

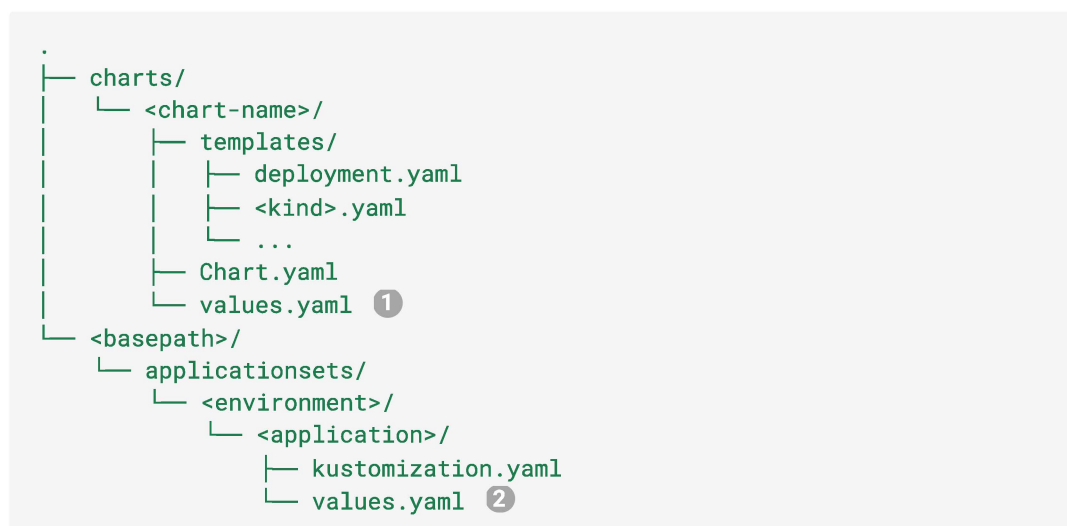
Man kan deploye Helm-charts ved hjelp av [HelmChartInflationGenerator](#). Denne kan man både bruke til å deploye public Helm charts og egne Helm-charts definert i ArgoCD repoet.

Egne Helm-charts kan man f.eks. lagre i `charts/<application>` i ArgoCD repoet.

For å deploye et Helm-chart kan man referere til det i

`<basepath>/applicationsets/<miljø>/<applikasjon>/kustomization.yaml`. I tillegg refererer man til en miljø-spesifikk `values.yaml` -fil på samme nivå i filstrukturen som `kustomization.yaml`.

Foreslått filstruktur:



- ① Overordnet `values.yaml` for Helm-chartet. Denne blir automatisk inkludert når man referer Helm-chartet i `kustomization.yaml` -filen.
- ② Miljøspesifikk `values.yaml` fil. Dersom du har hatt denne i f.eks. `charts/<application>/environment/<miljø>/values.yaml` tidligere kan du

kopiere filen til ny lokasjon.

I `kustomization.yaml` kan du bruke følgende mal:

```
apiVersion: kustomize.config.k8s.io/v1beta1
kind: Kustomization

helmGlobals:
  chartHome: ../../../../charts ❶

helmCharts: ❷
- name: <chart-name> ❸
  releaseName: <application>--<miljø>
  valuesFile: values.yaml ❹

resources:
- <kubernetes-manifest>.yaml ❺
```

- ❶ Relativ path til mappen hvor Helm-charts ligger fra `kustomization.yaml`. Eksempelet referer til `charts/`-mappe på rotnivå i ArgoCD repoet fra `<basepath>/applicationsets/<miljø>/<applikasjon>/kustomization.yaml`.
- ❷ Eksempelet viser minimum-konfigurasjon. Se [HelmChartInflationGenerator](#) for flere alternativer.
- ❸ Navn på Helm-chartet i `charts/`
- ❹ Relativ path fra `kustomization.yaml` fil til supplerende og miljøspesifikk YAML-fil. Overordnet `values.yaml` fra `charts/<application>/values.yaml` blir automatisk inkludert, så den trenger man ikke å referere.
- ❺ Valgfri seksjon. Dersom du har Kubernetes-manifester definert i samme mappe som `kustomization.yaml` må du inkludere disse her.

For å verifisere at manifestene dine er som forventet før du pusher til Git kan du bygge de lokalt ved å kjøre følgende kommando (krever at du har Helm og Kustomize installert):

```
kustomize build <relativ-path-til-mappe-med-kustomize-fil> --enable-helm
```

Migrere fra Application til ApplicationSet

Dersom du skal migrere fra Application til ApplicationSet burde det ikke by på særlig problemer. Det anbefales å gjøre det i et test-miljø først for å se at alt går som forventet.

1. Følg oppskrift over for å opprette relevante filer i `<basepath>/applicationsets/<miljø>/<applikasjon>`.
2. Slett ArgoCD Application.

3. Merge til main.
4. Flytt eventuelle Git tags som refereres i `targetRevision`.
5. Følg med på at deploy går som forventet.

Dersom ny deployment får nye navn på Kubernetes-ressurser kan gamle ressurser bli "orphaned". Dette problemet løser du enkelt ved å slette gammel deployment eller andre Kubernetes-ressurser som henger igjen. Dersom du benytter PVCer bør du være nøye på at disse blir opprettet med samme navn, slik at du ikke sletter data.

Deploy av vanilla Kubernetes-manifester (uten Helm eller Kustomize)

Hver mappe med manifester definert under `<miljø>/` vil opprette en Argo CD Application som deployer til dette miljøet.

For eksempel, for å deploye en `nginx` applikasjon sammen med noen secrets til `prod` i tenanten `uke-dokumentasjon` vil strukturen se slik ut:

```
├─ dokumentasjon
│   └─ applicationsets
│       └─ prod
│           └─ nginx
│               ├── deployment.yaml
│               ├── externalsecret.yaml
│               └── secretstore.yaml
└─ README.md
```

Dersom man har en `kustomization.yaml` fil i mappen er man nødt til å nevne alle Kubernetes-manifest-filer i resources.

Konfigurere syncPolicy for ApplicationSet

Offisiell dokumentasjon om [Sync Policy](#).

`ApplicationSet`-malen konfigurerer automatisk hvilken `syncPolicy` de genererte ArgoCD `Application` ene skal ha. Default-verdier for `syncPolicy` er:

```
syncPolicy:
  automated:
    prune: true
    selfHeal: true
    allowEmpty: true
```

I mange tilfeller vil det være ønskelig å beholde default-verdiene, fordi verdiene gjør at man automatisk får gjenspeilet ArgoCD-repoet uten behov for manuell oppfølging. Dette gjør at man blant annet har lavere risiko for at ressurser ligger igjen i OpenShift og bruker opp ressurser unødvendig.

I noen namespaces ønsker man å kjøre enda mer kontrollerte deploys og da er det greit å kunne overstyre `syncPolicy`. Dette gjør man i tenant-definisjon ved å sette:

```
argocd:
  syncPolicy:
    prune: <true/false>
    selfHeal: <true/false>
    allowEmpty: <true/false>
```

For mer informasjon om konfigurasjon av ArgoCD i tenant-definisjon, se [offisiell OpenShift dokumentasjon fra 2S](#).

Konfigurere Target revision

På samme måte som at man kan benytte [targetRevision i ArgoCD Application](#) for enklere deploy til test-miljøer (test/preprod) kan man også benytte dette i ApplicationSet.

Det anbefales å lese mer om hvordan targetRevision fungerer under [Kontinuerlig Deployment](#).

Det finnes også informasjon om bruk av [Custom auto-defined targetRevision](#) i Sopra Steria sin dokumentasjon.

Skru på auto-generert targetRevision i ApplicationSets

For å kunne benytte `targetRevision` for ApplicationSets i et miljø må man sette `custom_auto_defined_targetRevision: true` under ønsket environment i tenant-definisjonen.

```
...
environments:
  - name: test
    custom_auto_defined_targetRevision: true
  - name: preprod
    custom_auto_defined_targetRevision: true
argocd:
  enable_auto_defined_apps: true
...
```

NB! I prod anbefales det ikke å skru på autogenerated `targetRevision` for `ApplicationSets`. Dette fordi god GitOps praksis er å basere prod-miljøet på det som ligger i main.

Genererte verdier for targetRevision i ApplicationSets

Når man har skrudd på bruk av `targetRevision` for et miljø i `ApplicationSets`, vil `targetRevision` for for genererte ArgoCD applikasjoner automatisk få følgende format:

```
targetRevision: <application-navn>-<miljø>
```

Eksempel på genererte `targetRevision` verdier:

```
└─ <basepath>
  └─ applicationsets
    └─ test
      ├── namespace-secrets # targetRevision: namespace-secrets-test
      │   └─ ...
      └─ email-api # targetRevision: email-api-test
          └─ ...
    └─ preprod
      ├── namespace-secrets # targetRevision: namespace-secrets-preprod
      │   └─ ...
      └─ email-api # targetRevision: email-api-preprod
          └─ ...
```